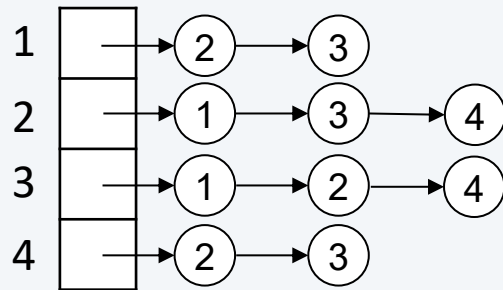


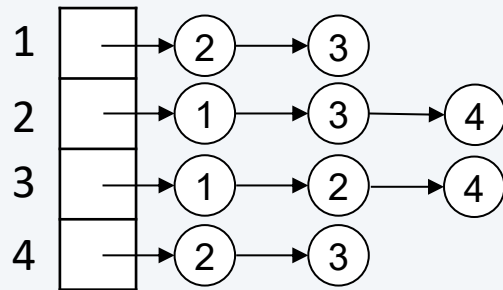
9 Graphen



9.1 Grundlagen

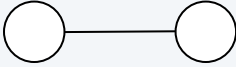
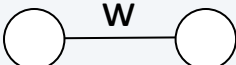
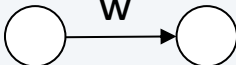
9.2 Gerichtete Graphen in Java

9 Graphen

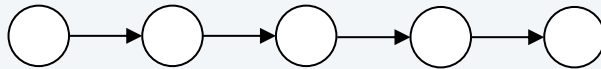


9.1 Grundlagen

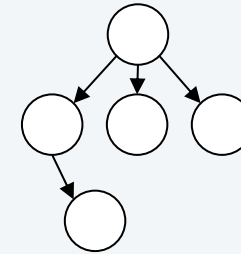
9.2 Gerichtete Graphen in Java

- Knoten  
- Kante, gerichtete Kante  
- gewichtete Kanten  

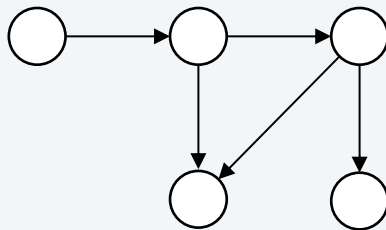
Verkettete Liste:



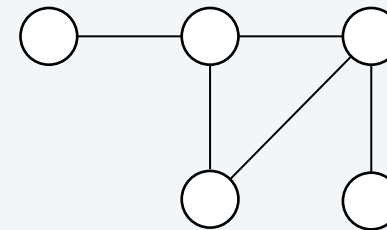
Baum:



Gerichteter Graph (*Digraph*)



Allgemeiner Graph

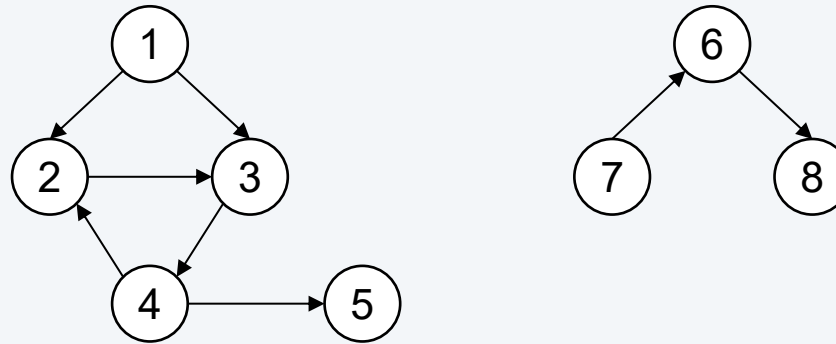


- **Graph.** $G = (V, E)$ ist eine Menge $V = \{1, 2, \dots, n\}$ von Knoten (*vertices*) und eine Menge $E \subseteq V \times V$ von Kanten (*edges*)
- **Ungerichteter Graph.** Jede Kante $\{v_1, v_2\}$ ist ein **ungeordnetes Paar** von Knoten; Kante verbindet zwei Knoten ohne Richtung vorzugeben
Beispiel: Wanderkarte (Knoten = Ausgangs- u. Zielpunkte, Gipfel, ...
Kanten = Wege)
- **Gerichteter Graph:** Jede Kante (v_1, v_2) ist ein **geordnetes Paar** von Knoten; Kante verbindet zwei Knoten in bestimmter Richtung
Beispiel: Produktionsplan (Abfolge der Produktionsschritte), Straßenkarte einer Stadt mit Einbahnstraßen, ...
- **Gewichteter Graph:** Jeder Kante $e \in E$ ist ein Wert $w(e) \in \mathbb{R}$ zugeordnet (Wert stellt je nach Anwendungsbereich Kosten, Dauer, Entfernung, ... dar)
Funktion $w: E \rightarrow \mathbb{R}, e \mapsto w(e)$ heißt die Gewichtung oder Bewertung von E
Beispiel: Straßenkarte, wobei Knoten = Orte, Kanten = Straßen und Gewichte = Entfernung zwischen den Orten

- Zwei Knoten v_1 und v_2 sind **benachbart** oder **adjacent**, wenn es eine ungerichtete Kante $\{v_1, v_2\}$ oder eine gerichtete Kante (v_1, v_2) gibt
- Grad (*degree*) eines Knotens v entspricht der Anzahl der Kanten, die diesen Knoten mit anderen Knoten verbinden
- Weg (*path*) in einem Graphen G von einem Knoten v_1 zu einem Knoten v_n ist die Folge von Knoten $v_1, v_2, \dots, v_n$ mit der Eigenschaft, dass $\{v_1, v_2\}, \{v_2, \dots\}, \{\dots, v_n\}$ Kanten von G sind; Weg = Liste von mit Kanten verbundenen Knoten
- Graph heißt **zusammenhängend** (*connected*), wenn von jedem Knoten zu jedem anderen Knoten ein Weg existiert
- Graph heißt **nicht zusammenhängend**, wenn er aus mindestens zwei Komponenten besteht; Graph heißt zusammenhängend, wenn er nur eine Komponente hat
- Weg heißt einfach, wenn kein Knoten mehrfach besucht wird
- Weglänge ist die Anzahl der Kanten auf dem Weg
- Zyklus ist ein Weg, bei dem Anfangs- und Endknoten gleich sind

- **Eingangsgrad** $InDeg(v)$ eines Knotens v ist Anzahl der Kanten, die in v enden
- **Ausgangsgrad** $OutDeg(v)$ eines Knotens v ist die Anzahl der Kanten, die von v ausgehen
- **Grad** $Deg(v)$ eines Knotens v ist die Summe aus Ausgangsgrad und Eingangsgrad: $Deg(v) = InDeg(v) + OutDeg(v)$
- **Quelle**: Knoten mit Eingangsgrad = 0 (gerichtete Graphen können mehrere Quellen besitzen)
- **Senke**: Knoten mit Ausgangsgrad = 0
- **Isolierter Knoten**: Knoten mit Eingangs- und Ausgangsgrad = 0
- Graphen, bei denen jeder Knoten denselben Grad r hat, nennt man **regulär** vom Grad r
- Graph heißt **vollständig**, wenn für jeden Knoten v gilt: $Deg(v) = |V| - 1$ d. h. je zwei Knoten sind genau durch eine Kante verbunden

Beispiel für einen Graphen



- $G = (V, E)$ mit zwei Komponenten, $V = \{ 1, 2, 3, 4, 5, 6, 7, 8 \}$
- $E = \{ (1, 2), (1, 3), (2, 3), (3, 4), (4, 2), (4, 5), (6, 8), (7, 6) \}$
- Quelle = Knoten 1 und 7, Senken = Knoten 5 und 8
- $\text{Deg}(1) = \text{OutDeg}(1) + \text{InDeg}(1) = 2 + 0 = 2$
- Weg $p(1, 5) = \text{Knotenfolge } (1, 3, 4, 5) \text{ oder } (1, 2, 3, 4, 5)$
- einfacher Weg $p(3, 5) = \text{Knotenfolge } (3, 4, 5)$
- Weglänge $| p(3, 5) | = 2$
- Zyklus = $(2, 3, 4, 2)$

- Knoten werden durch eindeutige Knotennummern (ganze Zahlen, z. B. aus dem Bereich von 0 bis $n - 1$, $n = |V|$) repräsentiert
- Weitere Information zu den Knoten (z. B. Namen) wird in eigener Datenstruktur (Abbildung von Knotennummer auf Information) verwaltet, im einfachsten Fall ein Feld von Objekten

```
String[] names = new String[n];
```

```
Person[] persons = new Person[n];
```

```
class Person {  
    String name;  
    ...  
}
```

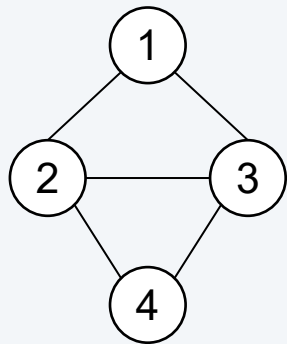
- Damit genügt es, Nachbarschaftsbeziehungen (Adjazenzbeziehungen) zwischen den Knoten darzustellen
- Zwei Möglichkeiten:
 - Adjazenzmatrix oder
 - Adjazenzliste

Adjazenzbeziehungen werden in **quadratischer Matrix** der Größe $n \times n$ mit $n = |V|$ repräsentiert:

- **Index** in beiden Dimensionen ist Knotennummer,
- **Elemente** sind boole'sche Werte bei ungewichteten oder numerische Werte bei gewichteten Graphen

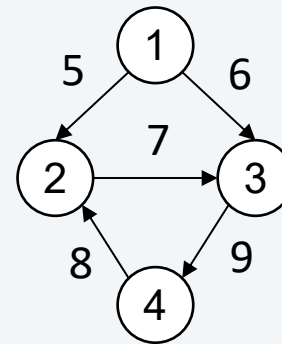
Vorteil: effizienter Zugriff, $O(1)$ Nachteil: hoher Speicherplatzbedarf, $O(n^2)$

Beispiele:



	1	2	3	4
1	F	T	T	F
2	T	F	T	T
3	T	T	F	T
4	F	T	T	F

Ungewichteter ungerichteter Graph



	1	2	3	4
1	0	5	6	0
2	0	0	7	0
3	0	0	0	9
4	0	8	0	0

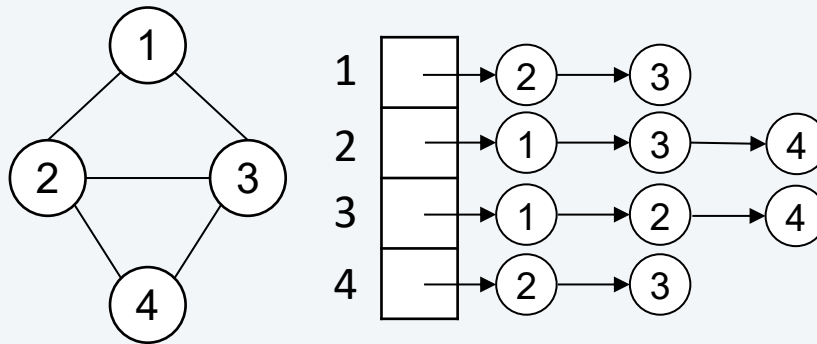
Gewichteter gerichteter Graph

Adjazenzbeziehungen werden in einem Feld der Länge $n = |V|$ repräsentiert:

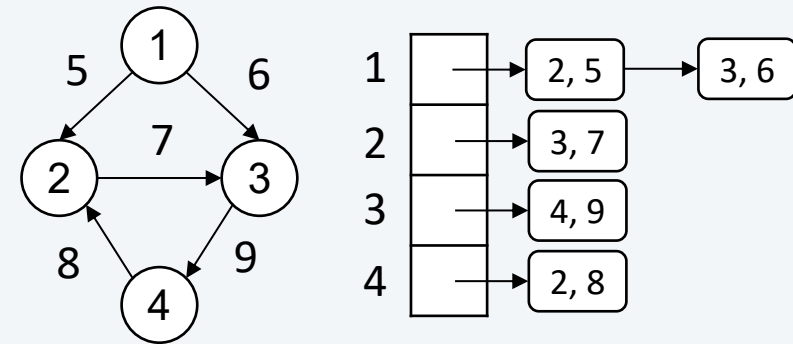
- **Index** ist Knotennummer; in jedem Feldelement ankert Liste aller Knoten, die mit dem Indexknoten verbunden sind
- **Listenelemente** enthalten Knotennummer und bei gewichteten Graphen numerischen Wert

Vorteil: geringer Speicherbedarf, $O(|E|)$ Nachteil: langsamer Zugriff, $O(|E|/n)$

Beispiele:

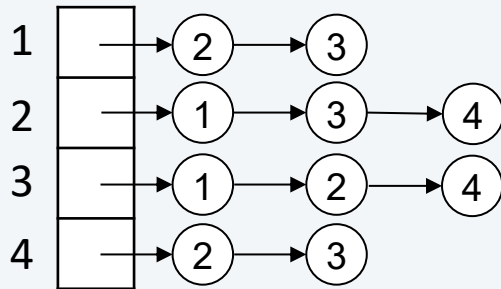


Ungewichteter ungerichteter Graph



Gewichteter gerichteter Graph

9 Graphen



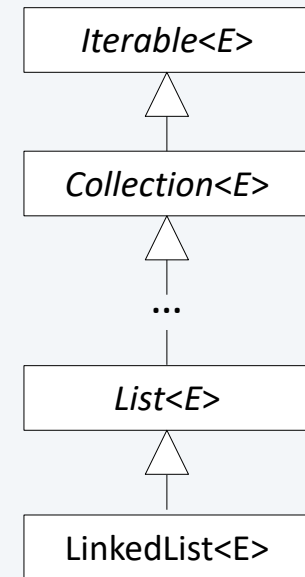
9.1 Grundlagen

9.2 Gerichtete Graphen in Java

Schnittstelle Graph für ungewichtete Graphen:

```
public interface Graph {  
    int getNrOfVertices();  
    int getNrOfEdges();  
    void addEdge(int v, int w);  
    void removeEdge(int v, int w);  
    Iterable<Integer> getAdj(int v);  
    int degree(int v);  
}
```

an v angrenzende Knoten



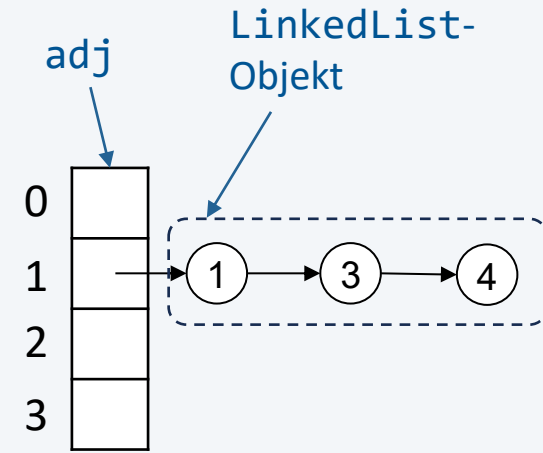
Schnittstelle Digraph für ungewichtete gerichtete Graphen:

```
public interface Digraph extends Graph {  
    int outdegree(int v);  
    int indegree(int v);  
    Digraph reverse();  
}
```

Klasse AdjListDigraph :

```
public class AdjListDigraph implements Digraph {  
    private int nrOfVertices;  
    private int nrOfEdges;  
    private List<Integer>[] adj;  
    private int[] indegree;   
  
    public AdjListDigraph(int nrOfVertices) {  
        this.nrOfVertices = nrOfVertices;  
        this.nrOfEdges = 0;  
        this.adj = (List<Integer>[]) new List<?>[nrOfVertices];  
        for (int v = 0; v < nrOfVertices; v++) {  
            this.adj[v] = new LinkedList<Integer>();  
        }  
        this.indegree = new int[nrOfVertices];  
    }  
  
    public int getNrOfVertices() { return nrOfVertices; }  
    public int getNrOfEdges() { return nrOfEdges; }  
    ...  
}
```

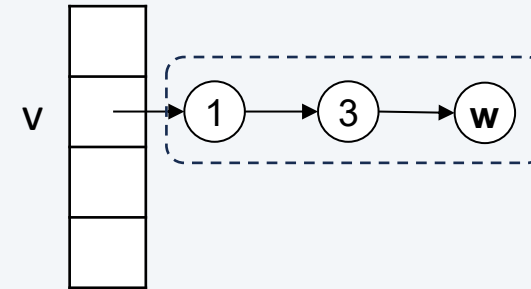
Eingangsgrad $InDeg(v)$
eines Knotens v



Methoden addEdge und removeEdge:

```
public void addEdge(int v, int w) {  
    validateVertex(v);  
    validateVertex(w);  
    adj[v].add(w);  
    indegree[w]++;  
    nrOfEdges++;  
}
```

```
public void removeEdge(int v, int w){  
    validateVertex(v);  
    validateVertex(w);  
    if (adj[v].remove((Integer) w)) {  
        indegree[w]--;  
        nrOfEdges--;  
    }  
}
```



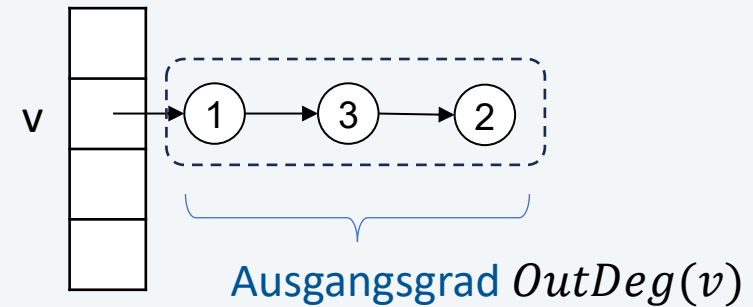
```
private void validateVertex(int v) {  
    if (v < 0 || v >= nrOfVertices) {  
        throw new IllegalArgumentException(...);  
    }  
}
```

Methoden degree, outdegree und indegree:

```
public int outdegree(int v) {  
    validateVertex(v);  
    return adj[v].size();  
}
```

```
public int indegree(int v) {  
    validateVertex(v);  
    return indegree[v];  
}
```

```
public int degree(int v) {  
    validateVertex(v);  
    return outdegree(v) + indegree(v);  
}
```

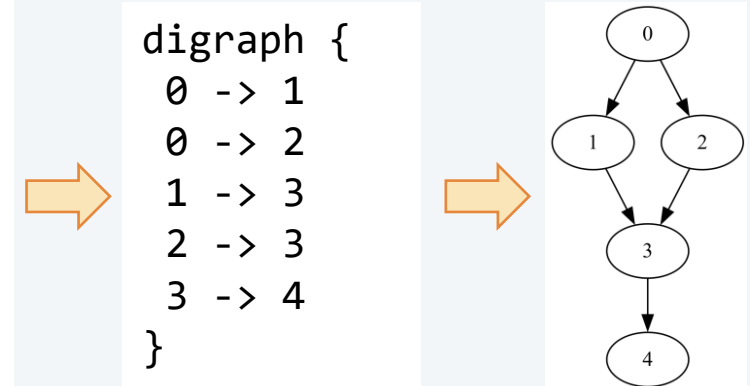


Methoden reverse und toString:

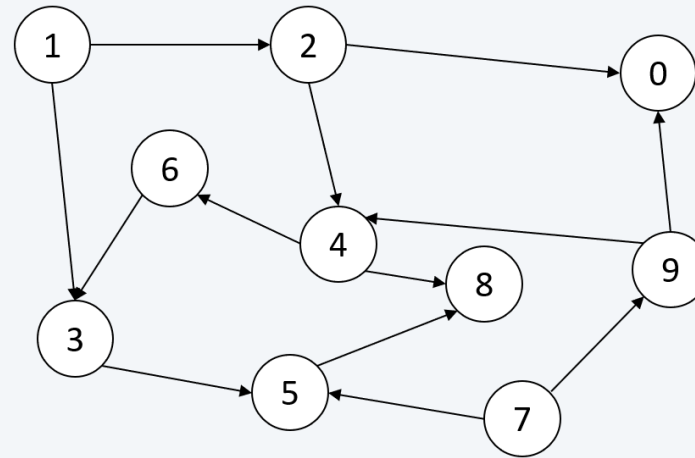
```
public Digraph reverse() {  
    Digraph r = new AdjListDigraph(nrOfVertices);  
    for (int v = 0; v < nrOfVertices; v++) {  
        for (int w : getAdj(v)) {  
            r.addEdge(w, v);  
        }  
    }  
    return r;  
}
```

this: r:
(v, w) → (w, v)

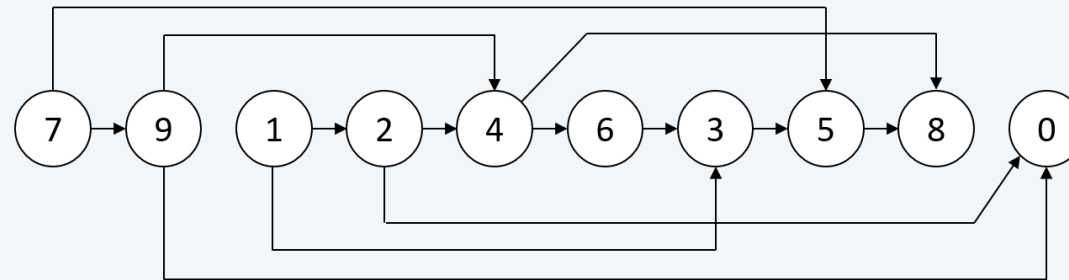
```
public String toString() {  
    StringBuilder sb = new StringBuilder();  
    sb.append("digraph {\n");  
    for (int v = 0; v < nrOfVertices; v++) {  
        for (int w : adj[v]) {  
            sb.append(" %d -> %d\n".formatted(v, w));  
        }  
    }  
    sb.append("}");  
    return sb.toString();  
}
```



- Teilweise Ordnungen können wir als **gerichtete Graphen** darstellen



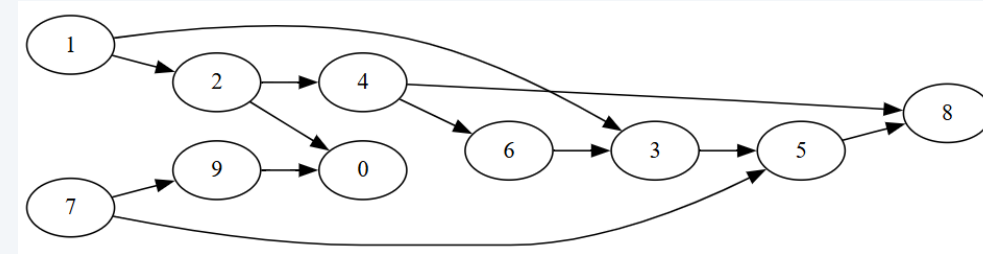
- Das Problem des topologischen Sortierens besteht darin, die teilweise Ordnung in eine lineare Ordnung einzubetten, z. B.



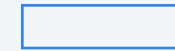
Wahl der Datenstruktur

```
Digraph g = new AdjListDigraph(10);  
while (...) {  
    int v = ... ;  
    int w = ... ;  
    g.addEdge(v, w);  
}
```

```
digraph {  
  1 -> 2  
  1 -> 3  
  2 -> 4  
  2 -> 0  
  3 -> 5  
  4 -> 6  
  4 -> 8  
  4 -> 0  
  5 -> 8  
  6 -> 3  
  7 -> 5  
  7 -> 9  
  9 -> 0  
}
```



Lösungsidee für das topologische Sortieren:



- Füge alle Knoten ohne Vorgänger ($\text{InDeg}(v) = 0$) zu einer Liste `srcNodes` hinzu
- Für alle Knoten in der Liste `srcNodes`:
 - Entferne Knoten `v` aus der Liste `srcNodes` und füge den Knoten zu Ergebnis hinzu
 - Vermindere für jeden Nachfolger von `v` den Zähler der noch verbleibenden Vorgänger. Wird dieser Zähler 0, dann füge diesen Nachfolger in die Liste der Knoten ohne Vorgänger ein.

```
public static int[] sort(Digraph g) {  
    int[] result = new int[g.getNrOfVertices()];  
    int i = 0;  
    int[] indegree = new int[g.getNrOfVertices()];  
    SequencedSet<Integer> srcNodes = new TreeSet<>();  
    for (int v = 0; v < g.getNrOfVertices(); v++) {  
        indegree[v] = g.indegree(v);  
        if (indegree[v] == 0) {  
            srcNodes.add(v);  
        }  
    }  
    while (!srcNodes.isEmpty()) {  
        int v = srcNodes.removeFirst();  
        result[i] = v; i++;  
        for (int w : g.getAdj(v)) {  
            indegree[w]--;  
            if (indegree[w] == 0) {  
                srcNodes.add(w);  
            }  
        }  
    }  
    return result;  
}
```

← Sammle alle Knoten
ohne Vorgänger

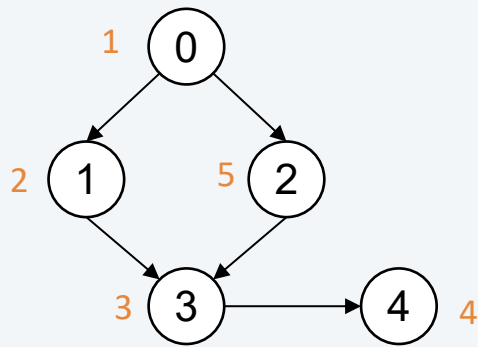
← füge Knoten zu
Ergebnis hinzu

← vermindere die Zähler
der Vorgänger

← Sammle Knoten ohne
Vorgänger

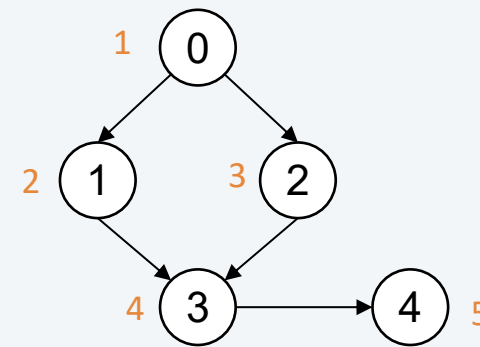
- Alle Knoten in einer bestimmten Reihenfolge durchwandern
- Nach Knoten in einem Graphen in einer bestimmten Reihenfolge suchen

Depth First Order (Tiefensuche)



Reihenfolge: 0, 1, 3, 4, 2

Breadth First Order (Breitensuche)



Reihenfolge: 0, 1, 2, 3, 4

Depth First Order (Tiefensuche)

```
public class DepthFirstOrder {
    private final boolean[] marked;

    public DepthFirstOrder(Graph g) {
        this.marked = new boolean[g.getNrOfVertices()];
        for (int v = 0; v < g.getNrOfVertices(); v++) {
            if (!marked[v]) {
                dfs(g, v);
            }
        }
    }

    private void dfs(Graph g, int v) {
        marked[v] = true;
        for (int w : g.getAdj(v)) {
            if (!marked[w]) {
                dfs(g, w);
            }
        }
    }
}
```

Depth First Order (Tiefensuche)

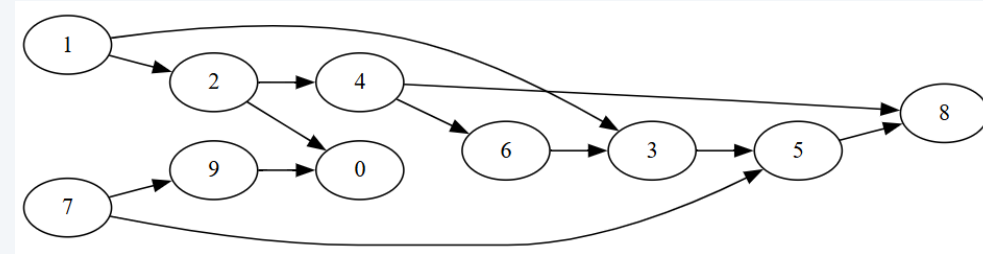
```
public class DepthFirstOrder {  
    private final boolean[] marked;  
    private List<Integer> preOrder;  
    private List<Integer> postOrder;  
  
    public DepthFirstOrder(Graph g) {  
        this.preOrder = new ArrayList<Integer>();  
        this.postOrder = new ArrayList<Integer>();  
        ...  
    }  
    private void dfs(Graph g, int v) {  
        preOrder.add(v);  
        marked[v] = true;  
        for (int w : g.getAdj(v)) {  
            if (!marked[w]) {  
                dfs(g, w);  
            }  
        }  
        postOrder.add(v);  
    }  
}
```

Depth First Order (Tiefensuche)

Anwendung

```
Digraph g = new AdjListDigraph(10);  
while (...) {  
    int v = ... ;  
    int w = ... ;  
    g.addEdge(v, w);  
}
```

```
digraph {  
  1 -> 2  
  1 -> 3  
  2 -> 4  
  2 -> 0  
  3 -> 5  
  4 -> 6  
  4 -> 8  
  5 -> 8  
  6 -> 3  
  7 -> 5  
  7 -> 9  
  9 -> 0  
}
```



```
DepthFirstOrder dfo = new DepthFirstOrder(g);  
List<Integer> pre = dfo.preOrder();  
IO.println("Preorder: " + pre);
```

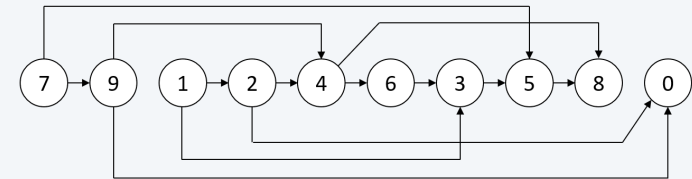
Preorder: [0, 1, 2, 4, 6, 3, 5, 8, 7, 9]

```
List<Integer> post = dfo.postOrder();  
IO.println("Postorder: " + post);
```

Postorder: [0, 8, 5, 3, 6, 4, 2, 1, 9, 7]

```
List<Integer> reversePost = post.reversed();  
IO.println("Reverse postorder: "+reversePost);
```

Reverse postorder: [7, 9, 1, 2, 4, 6, 3, 5, 8, 0] ← topologisch sortierte Knotenfolge!





UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Studiengang Software Engineering
Fakultät für Informatik, Kommunikation, Medien
Campus Hagenberg

9 Graphen

9.1 Grundlagen

9.2 Gerichtete Graphen in Java